Introdução ao

Sandra Mota

Conteúdo

Informações gerais 3

Bases de Dados 5

Tabelas 6

Registos 10

Visualizações 11

Complementos de Comandos 17

Tabelas Sumárias 20

Estes comandos são maioritariamente relativos ao MySql, outros sistemas podem usar variações a estes comandos. Algumas diferenças estão identificadas deste modo:

As informações *sublinhadas e em itálico* devem ser substituídas pelo correspondente na base de dados a utilizar.

Informações gerais

- tipos de dados:
  - int – nºs inteiros;
  - real – nº reais (ex: temperatura, medidas simples)
  - float – nºs reais mais precisos (ex: cálculos cientificos, estatísticas);
  - decimal(*p*,*s*) / numeric(*p*,*s*) – casas decimais predefinidas (p-nº total de dígitos, s- nº de dígitos após a vírgula);
  - char(*tamanho*) – caracteres (tamanho fixo; 0-255);
  - varchar(*tamanho*) – texto (tamanho variável; 0-65.535);
  - text – texto;
  - date – datas (AAAA-MM-DD);
  - year – anos (YYYY ou YY; só aceita entre 1901 e 2155);
  - time – horas (HH:MM:SS);
  - datetime – data e hora (AAAA-MM-DD HH:MM:SS);
  - boolean (true/false/null);

- operadores de comparação:

=	Igual
>	Maior que
<	Menor que
>=	Maior oi igual a
<=	Menor ou igual a
<>	Diferente (em algumas versões pode ser !=)
BETWEEN	Dentro de um intervalo
LIKE	Procurar por um padrão
IN	Para especificar múltiplos valores possíveis para uma coluna

- operadores aritméticos:

+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Módulo

- operadores bitwise:

&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR

- operadores compostos:

+=	Adição e atribuição
-=	Subtração e atribuição
*=	Multiplicação e atribuição
/=	Divisão e atribuição
%=	Módulo e atribuição
&=	Bitwise AND e atribuição
|=	Bitwise OR e atribuição
^=	Bitwise exclusive OR e atribuição

- operadores lógicos:

ALL	True se todos os valores da subconsulta satisfizerem a condição
AND	True se todas as condições separadas por AND forem true
ANY / SOME	True se algum dos valores da subconsulta satifizerem a condição
BETWEEN	True se o operando estiver dentro do intervalo de comparação
EXISTS	True se a subconsulta retornar 1 ou mais registos
IN	True se o operando é igual a um na lista de expressões
LIKE	True se o operando corresponde a um padrão
NOT	Retorna um registo se a condição é not true
OR	True se uma das condições separadas por OR for true

Bases de Dados

- criar base de dados:
  - CREATE DATABASE *nomeBaseDados*;
    - ex: create database agendaT;

- mostrar base de dados existentes:
  - SHOW DATABASES;

- trabalhar na base de dados escolhida
  - USE *nomeBaseDados*;
    - ex: use agendaT;

- saber qual a base de dados escolhida
  - SELECT DATABASE();

- criar backup da BD:
  - BACKUP DATABASE *nomeBaseDados* TO DISK='*caminho*';
    - ex: backup database vendas to disk='D:\backups\DBVendas.bak';
  - backup apenas das partes alteradas desde o último backup
    - BACKUP DATABASE *nomeBaseDados* TO DISK='*caminho*' WITH DIFFERENTIAL;

- eliminar base de dados:
  - DROP DATABASE *nomeBaseDados*;

Tabelas

▶ criar tabela:

▷ CREATE TABLE *nomeTabela* (*campo1 tipoDados*(*tamanho*), *campo2 tipoDados*(*tamanho*), *campo3 tipoDados*(*tamanho*));

- ex: create table nomes (nomes_ID integer, nomes_nome varchar(40), nomes_local varchar(40));

↳ **restrições**: para definir restrições basta acrescentar após o nº de caracteres (PK pode ser definida no final)

- NOT NULL – assegura que o campo não pode ter valores nulos
  - ex: create table pessoas (ID integer **not null**, apelido varchar(255) **not null**, nome varchar(255), idade integer, primary key (ID));
- UNIQUE – assegura que todos os valores do campo são diferentes
- PRIMARY KEY – identifica univocamente cada registo da tabela (por defeito já é NOT NULL e UNIQUE; pode ser insrido junto do campo ou no final)
  - ex: create table alunos (id int **primary key** auto_increment, nome varchar(100), data_nascimento date, curso_id int, foreign key (curso_id) references cursos(id));
  - ex: create table pessoas (ID integer not null, apelido varchar(255) not null, nome varchar(255), idade integer, **primary key** (ID));
- AUTO_INCREMENT (não é uma restrição, mas um atributo)
  - permite que um número único seja automaticamente gerado quando um novo registo é inserido, normalmente associado à PK; só pode ser definido para 1 campo por tabela; por defeito começa em 1 e acresce +1 para cada novo registo; caso seja manualmente inserido um valor superior ao último existente, os novos serão gerados a partir do inserido de forma manual.
    - ex: create table alunos (id int primary key **auto_increment**, nome varchar(100), data_nascimento date, curso_id int, foreign key (curso_id) references cursos(id));
- FOREIGN KEY – relaciona tabelas e impede ações que destruiriam esas relações
  - FOREIGN KEY (*campoTabela*) REFERENCES *nomeTabelaReferência*(*campoTabelaReferência*);
    - ex: create table alunos (id int primary key auto_increment, nome varchar(100), data_nascimento date, curso_id int, **foreign key** (curso_id) **references** cursos(id));
- CHECK – assegura que os valores do campo satisfazem uma condição específica
  - ex: create table clientes (id int not null, nome varchar(255), apelido varchar(255), idade int **check (idade >=18)**);
- DEFAULT – estabelece um valor padrão para o campo caso não seja especificado um valor
  - ex: se uma empresa tem a maioria de clientes em Lisboa, pode ficar definido por defeito e só alterar na inclusão dos clientes diferenciados
    - ex: create table clientes (id int not null, nome varchar(255), apelido varchar(255), idade int, cidade varchar(255) **default 'lisboa'**);
  - ex: preencher com dia/hora atual
    - ex: create table pedidos (id int nut null, pedido int not null, data_pedido date **default getdate()**);

↳ podemos **nomear e definir uma restrição para múltiplas colunas** (facilita se quisermos modificar ou eliminar as restrições em conjunto posteriormente) – funciona com PRIMARY KEY, FOREIGN KEY*, UNIQUE e CHECK**)

- CREATE TABLE *nomeTabela* (*campo1* *tipoDados*(*tamanho*), *campo2* *tipoDados*(*tamanho*), *campo3* *tipoDados*(*tamanho*) CONSTRAINT *nome_restrição* *restrição* (*campo1*,*campo2*,…));
  - No caso da PK podemos criar restrição com nome para definir vários campos como PK. A PK é só uma (o nome que lhe atribuirmos) mas é constituída por vários campos
- *FK: CREATE TABLE *nomeTabela* (*campo1* *tipoDados*(*tamanho*), *campo2* *tipoDados*(*tamanho*), *campo3* *tipoDados*(*tamanho*) CONSTRAINT *nome_restrição* FOREIGN KEY (*campoTabela*) REFERENCES *nomeTabelaReferência*(*campoTabelaReferência*);
- **CHECK: CREATE TABLE *nomeTabela* (*campo1* *tipoDados*(*tamanho*), *campo2* *tipoDados*(*tamanho*), *campo3* *tipoDados*(*tamanho*) CONSTRAINT *nome_restrição* CHECK (*condições*));

▶ criar tabela a partir de tabelas existentes (não copia PK):

▷ CREATE TABLE AS (copia dados de uma tabela existente para uma nova tabela que cria)

- CREATE TABLE *nomeNovaTabela* AS SELECT *campo1*, *campo2*, … FROM *nomeTabelaExistente* [WHERE *condição*];
  - ex: create table tabela_teste as select nome_cliente, contacto from clientes;

Não suportado em MySQL

▷ SELECT INTO (copia dados de uma tabela existente para uma nova tabela que cria)

- toda a tabela:
  - SELECT * INTO *nomeNovaTabela* [IN *BDexterna*] FROM *nomeTabelaExistente* [WHERE *condição*];
    - ex: select * into backupclientes in 'backup.mdb' from clientes;
- alguns campos:
  - SELECT *campo1*, *campo2*, … INTO *nomeNovaTabela* [IN *BDexterna*] FROM *nomeTabelaExistente* [WHERE *condição*];
    - ex: select nome_cliente, contato into backupclientes from clientes;
- pode ser usado para criar uma nova tabela vazia com a mesma estrutura da antiga, usando o WHERE para não retornar dados)
  - ex: select * into clientes2025 from clientes where 1=0;
- de várias tabelas:
  - SELECT *nomeTabela.campo*, *nomeTabela.campo*, … INTO *nomeNovaTabela* FROM *nomeTabela1* JOIN *nomeTabela2* ON *nomeTabela1.campo* = *nomeTabela2.campo*;
    - ex: select clientes.nome_cliente, pedidos.id_pedido into backup_pedidos_clientes from clientes join pedidos on clientes.id_cliente=pedidos.id_cliente;

▶ mostrar tabelas existentes na base de dados:

▷ SHOW TABLES;

▶ eliminar tabela:

▷ DROP TABLE *nomeTabela*;

▶ eliminar dados da tabela:

▷ TRUNCATE TABLE *nomeTabela*;

- mostrar campos da tabela:
  - DESCRIBE *nomeTabela*;
    - ex: describe nomes;

- alterar tabela – alter – **Atenção**: Pode afetar dados existentes. Algumas alterações exigem que a tabela esteja vazia ou que não haja dependências.
  - ALTER TABLE *nomeTabela* *ação*;
    - adicionar campo
      - ALTER TABLE *nomeTabela* ADD *campo* *tipoDados*;
        - ex: alter table clientes add data_nascimento date;
    - remover campo
      - ALTER TABLE *nomeTabela* DROP *campo*;
        - ex: alter table clientes drop apelido;
    - adicionar restrição (sem nome)
      - ALTER TABLE *nomeTabela* ADD *restrição* (*campo*);
        - ex: alter table clientes add unique (id_cliente);
      - **exceções**:
        - FOREIGN KEY: ALTER TABLE *nomeTabela* ADD FOREIGN KEY (*campoTabela*) REFERENCES *nomeTabelaReferência*(*campoTabelaReferência*);
        - DEFAULT: ALTER TABLE *nomeTabela* ALTER *campo* SET DEFAULT '*valorDefault*';
    - adicionar restrição (com nome)
      - ALTER TABLE *nomeTabela* ADD CONSTRAINT *nomeRestrição* *restrição* (*campo*);
        - ex: alter table clientes add constraint uc_clientes unique (id_cliente, apelido);
      - **exceções:**
        - FOREIGN KEY
          - ALTER TABLE *nomeTabela* ADD CONSTRAINT *nome_restrição* FOREIGN KEY (*campoTabela*) REFERENCES (*campoTabelaReferência*);
        - CHECK:
          - ALTER TABLE *nomeTabela* ADD CONSTRAINT *nome_restrição* CHECK (*condições*);
    - remover restrição
      - PRIMARY KEY (sem nome)
        - ALTER TABLE *nomeTabela* DROP PRIMARY KEY;
      - FOREIGN KEY (sem nome)
        - 1º temos que perceber que nome foi gerado automaticamente para a FK (a 1ª forma retorna apenas o nome, a 2ª indica também qual o campo da FK e a que tabela faz referência):
          - SELECT CONSTRAINT_NAME FROM information_schema.TABLE_CONSTRAINTS WHERE TABLE_NAME = '*nomeTabela*' AND CONSTRAINT_TYPE = 'FOREIGN KEY';

            ou
          - SELECT CONSTRAINT_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_NAME = '*nomeTabela*' AND REFERENCED_TABLE_NAME IS NOT NULL;

        - remoção:
            - ALTER TABLE *nomeTabela* DROP FOREIGN KEY *nomeFK*;
    - NOT NULL
        - ALTER TABLE *nomeTabela* MODIFY *campo* *tipoDados* NULL;
            - ex: alter table clientes modify nome_cliente varchar(50) null;
    - restrições com nome (PRIMARY KEY, UNIQUE*, CHECK, DEFAULT*):
        - ALTER TABLE *nomeTabela* DROP CONSTRAINT *nomeRestrição*;
            - ex: alter table clientes drop constraint chk_idade;
        - *exceções:
            - UNIQUE
                - ALTER TABLE *nomeTabela* DROP INDEX *nomeRestrição*;
                - ex: alter table clientes drop índex uc_clientes;
            - DEFAULT
                - ALTER TABLE *nomeTabela* ALTER *campo* DROP DEFAULT;
    - restrições em 1 campo (UNIQUE, CHECK, DEFAULT):
        - ALTER TABLE *nomeTabela* DROP *restrição* *campo*;
            - ex: alter table clientes drop check nome_cliente;
- alterar o tipo de um campo (pode ser usado para alterar restrições)
    - ALTER TABLE *nomeTabela* MODIFY *campo* *novoTipoDados* [*restrições*];
        - ex: alter table vendas modify preço_unitario decimal(10,2);
        - ex: alter table pessoas modify idade int not null;

SQL Server / MS Access
- ALTER TABLE *nomeTabela* ALTER COLUMN *campo* *tipoDados* [*restrições*];

- alterar o valor do AUTO_INCREMENT (já tem de existir previamente)
    - ALTER TABLE *nomeTabela* AUTO_INCREMENT=*número*;
- renomear campo
    - ALTER TABLE *nomeTabela* RENAME COLUMN *nomeCampoAntigo* TO *nomeCampoNovo*;
        - ex: alter table clientes rename column nome to cliente;
    - ALTER TABLE *nomeTabela* CHANGE *nomeCampoAntigo* *nomeCampoNovo* *tipoDados*; ▶ permite alteração do tipo de dados se necessário
        - ex: alter table produtos change nome_produto descricao varchar(255);

SQL Server
- EXEC sp_ename *'nomeTabela.nomeCampoAntigo'*, *'nomeCampoNovo'*, 'COLUMN';

▶ renomear tabela
- ▷ rename table *nomeTabelaAntigo* to *nomeTabelaNovo*;
    - ex: RENAME TABLE clientes TO clientes_arquivados;

▶ Índice
- ▷ Índices são usados para consultar dados da base de dados mais rapidamente. Os usuários não podem ver os índices, eles são usados apenas para agilizar pesquisas/consultas.
    - Criar:
        - CREATE INDEX *nomeIndice* ON *nomeTabela* (*campo1*, *campo2*, …);
    - Criar índice que não permite valores duplicados:
        - CREATE UNIQUE INDEX *nomeIndice* ON *nomeTavela* (*campo1*, *campo2*, …);
    - Remover
        - DROP INDEX *nomeIndice* ON *nomeTabela*;

Registos

- inserir registos:
  - todos os campos (incluindo ID mesmo se auto_increment):
    - INSERT INTO *nomeTabela* VALUES (*dadosCampo1*, *dadosCampo2*, *dadosCampo3*);
  - Alguns campos (não é necessário ID se auto_increment):
    - INSERT INTO *nomeTabela* (*campo1*, *campo2*, *campo3*) VALUES (*dadosCampo1*, *dadosCampo2*, *dadosCampo3*);
      - ex 1 registo: insert into nomes (nomes_ID, nomes_nome, nomes_local) values(10,'Sandra Mota', 'Lisboa');
      - ex múltiplos registos: insert into nomes (nomes_id, nomes_nome, nomes_local) values (6, 'Paulo Andrade', 'Aveiro'), (7, 'Sandra Costa', 'Leiria'), (8, 'Marcos Pereira', 'Setubal'), (9, 'Tania Ribeiro', 'Viseu');
- copiar dados de uma tabela e inserir noutra (os tipos de dados das duas tabelas devem ser correspondentes):
  - todos os campos:
    - INSERT INTO *nomeTabela2* SELECT * FROM *nomeTabela1* [WHERE *condição*];
  - alguns campos:
    - INSERT INTO *nomeTabela2* (*campo1*, *campo2*, …) SELECT *campo1*, *campo2*, … FROM *nomeTabela1* [WHERE *condição*];

- apagar registo:
  - DELETE FROM *nomeTabela* WHERE *condição*;
    - ex: delete from nomes where nomes_ID=13
- apagar todos os registos de uma tabela (mantendo estrutura, atributos e indexes)
  - DELETE FROM *nomeTabela*;

- editar registo 1 ou mais campos (pode ser utilizado para substituir todos os valores comuns de um campo, ou seja, se usarmos como condição um id só vai editar esse registo, mas se usarmos um dado comum a vários registos vai editar todos):
  - 1 campo:
    - UPDATE *nomeTabela* SET *campoAalterar*=*NovoValor* WHERE *condição*
      - ex: editar um registo (id 10)
        - ex: update nomes set nomes_local=Porto' where nomes_id=10;
      - ex: editar vários registos (todos os registos Lisboa)
        - ex: update nomes set nomes_local='Porto' where nomes_local='Lisboa';
  - vários campos:
    - UPDATE *nomeTabela* SET *campo1Aalterar*=*NovoValor*, *campo2Aalterar*=*NovoValor* WHERE *condição*
  - ex: editar um registo (id 10)
- ex: update nomes set nomes_local=Porto', duracao=3 where nomes_id=10;

Visualizações

▶ visualizar registos
- ▷ tabela toda:
  - SELECT * FROM *nomeTabela*;
    - ex: select * from nomes;
- ▷ um (ou mais) campo(s):
  - SELECT *campo1* (, *campo2, campo3*….) FROM *nomeTabela*;
    - ex: select nomes_id, nomes_nome from nomes;
- ▷ também podem conter condições para afinar pesquisa
  - SELECT *campo1* FROM *nomeTabela* where *condição*;
    - ex: select * from disciplinas where nome='design digital';

▶ visualizar apenas os registos diferentes dos campos referidos
- ▷ SELECT DISTINCT *campo1*, *campo2* FROM *nomeTabela*;
  - ex: select distinct país from clientes; →vai dar a listagem de todos os países na BD (sem duplicar)

▶ visualizar registos ordenados (podem ser usados múltiplos campos (*campo1*, *campo2*, …):
- ▷ ascendente (colocar o asc é facultativo, dado que por defeito é ascendente):
  - SELECT * FROM *nomeTabela* ORDER BY *campo* ASC;
    - ex: select * from alunos order by nome;
- ▷ descendente:
  - SELECT * FROM *nomeTabela* ORDER BY *campo* DESC;
    - ex: select * from alunos order by nome desc;
- ▷ ascendente e descentende:
  - ex: select * from clientes order by país asc, nome desc;

▶ visualizar com um limite de registos (TOP)
- SELECT * FROM *nomeTabela* ORDER BY *campo* DESC LIMIT *número*;
  - ex: select * from alunos order by nota desc limit 3;
- SELECT * FROM *nomeTabela* LIMIT *número*;
- SELECT *campo(s)* FROM *nomeTabela* WHERE condição LIMIT *número*;

SQL Server / MS Access
- SELECT TOP *number|percent column_name(s)* FROM *table_name* WHERE *condition*;

Oracle 12
- SELECT column_name(s) FROM table_name ORDER BY *column_name(s)* FETCH FIRST *number* ROWS ONLY;

Oracle mais antigo
- SELECT *column_name(s)* FROM *table_name* WHERE ROWNUM <= *number*;
- SELECT * FROM (SELECT *column_name(s)* FROM *table_name* ORDER BY *column_name(s)*) WHERE ROWNUM <= *number*;

▶ procurar por um padrão – LIKE (oposto NOT LIKE, normalmente usado com wildcards)
- ▷ SELECT * FROM *nomeTabela* WHERE *campo* LIKE *condição*;
  - ex: começa por A
- ▷ ex: select * from alunos where nome like 'A%';

▶ wildcards (podem ser usados ambos na mesma consulta)

▷ **%** (normalmente usado com o LIKE, representa qualquer sequência de caracteres, utilizado para consultas de texto como começa por, termina em, ou contém)

- ex: começa por A
  - select * from clientes where nome like 'A%'
- ex: termina em son
  - select * from clientes where apelido like '%son'
- ex: contém mar
  - select * from clientes where nome like '%mar%'

▷ **_** (o mesmo que o % mas substitui apenas 1 caractere)

- Ex: procurar código com 5 caracteres em que o 3º é A (encontra dados como XWAVC ou 12A34)
  - Select * from produtos where código like '__A__'

[ ] : representa algum caractere dentro dos parentesis retos *

- ex: select * from clientes where nome_cliente like '[bsp]%'; → retorna nomes começados por b, s ou p

^ : representa algum dos caracteres que não está dentro dos parêntesis retos *

- : representa algum caractere dentro de um intervalo específico *

- ex: select * from clientes where nome_cliente like '[a-f]%': → retorna nomes começados por a, b, c, d, e ou f

{ } : representa algum caractere escapado **

* - não suportado em PostgreSQL e MySQL

** - só suportado em Oracle

Wildcards do Microsoft Access

Símbolo	Descrição	Exemplo
*	Representa zero ou mais caracteres	bl* encontra **bl**, **black**, **blue** e **blob**
?	Representa um único caractere	h?t encontra **hot**, **hat** e **hit**
[]	Representa qualquer caractere único dentro dos colchetes	h[oa]t encontra **hot** e **hat**, mas não **hit**
!	Representa qualquer caractere **que não esteja** nos colchetes	h[!oa]t encontra **hit**, mas não **hot** nem **hat**
-	Representa qualquer caractere único dentro de um intervalo especificado	c[a-b]t encontra **cat** e **cbt**
#	Representa qualquer caractere numérico único	2#5 encontra **205**, **215**, **225**, **235**, **245**, **255**, etc.

▶ procurar por comparação

▷ **!=** / **<>** - diferente de

- SELECT * FROM *nomeTabela* WHERE *campo* <> *condição*;
  - ex: select * from cursos where id <> 2;

▷ BETWEEN - em intervalos de números, texto ou datas (oposto NOT BETWEEN)

- números

        - SELECT * FROM *nomeTabela* WHERE *campo* BETWEEN *valorMenor* AND *valorMaior*;
            - ex: select * from notas where nota between 12 and 14;
    - texto
        - SELECT * FROM *nomeTabela* WHERE *campo* BETWEEN *valorMenor* AND *valorMaior* order by *campo*;
            - ex: select * from produtos where nome_produto between 'ananas' and 'cerejas' order by nome_produto;
    - datas
        - ex: select * from pedidos where data_pedido between '2025-01-01' and '2025-06-30';
▷ < / <= / > / >= - menor/menor ou igual / maior/maior ou igual
    - SELECT * FROM *nomeTabela* WHERE *campo* *operador* *valor*;
        - ex: select * from notas where nota <= 12.5
▷ IN (oposto NOT IN) - múltiplas opções (como OR)
    - SELECT * FROM *nomeTabela* WHERE *campo* IN (*valor1*, *valor2*, *valor3*):
        - ex: select * from livros where autor in ('José Saramago', 'Eça de Queirós');
    - **incorporado noutros comandos**
        - ex: retornar todos os clientes que têm um pedido na tabela pedidos
            - ex: select * from clientes where id_cliente in (select id_cliente from pedidos);

▶ funções de agregação (ignoram valores null, exceto count(*); podem acrescer where *condição*; podemos adicionar contas dentro dos parentesis)
▷ mínimo
    - SELECT MIN(*campo*) FROM *nomeTabela*;
        - ex: select min(nota) from notas;
▷ máximo
    - SELECT MAX(*campo*) FROM *nomeTabela*;
        - ex: select max(nota) from notas;
▷ média
    - SELECT AVG(*campo*) FROM *nomeTabela*;
        - ex: select avg(nota) from notas;
▷ soma
    - SELECT SUM(*campo*) FROM *nomeTabela*;
        - ex: select sum(nota) from notas;
▷ contar
    - SELECT COUNT(*campo*) FROM *nomeTabela*;
        - ex: select count(nota) from notas;
    - contar sem campo específico com condição - count(*)
        - ex: select count(*) from empréstimos where data_devolucao_prevista=data_devolucao_real;
▷ contar valores distintos (ex: contar quantos países existem na BD)
    - ex: select count(distinct país) from clientes;
▷ **mostrar com o nome pretendido** (ex: máximo)
    - SELECT *função*(*campo*) AS *nomePretendido* from *nomeTabela*;
        - ex: select max(nota) as maior_nota from notas;
▷ **com conta**
    - ex: sabendo que cada artigo custa 10€ podemos saber o valor total somando a quantidade e multiplicando por 10; ou indo buscar essa info a outra tabela
        - ex: select sum(quantidade * 10) from pedidos;

- ex: select sum(preço * quantidade) from pedidos dá-me um exemplo produtos on pedidos.id_produto=produtos.id_produto;

▷ incorporados noutros comandos

- ex: retornar todos os produtos com preço maior que a média
  - ex: select * from produtos where preço > (select avg(preço) from produtos);

▶ visualização com dados de várias tabelas

▷ JOIN (ou INNER JOIN) – retorna registos comuns a ambas as tabelas

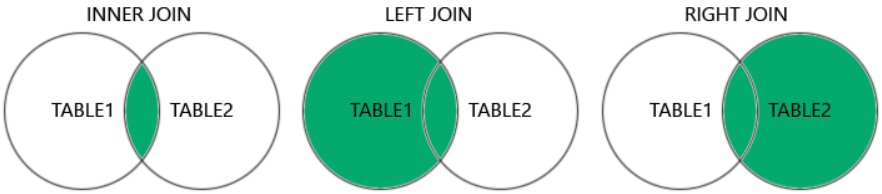
- SELECT *nomeTabela.campoAVisualizar*, *nomeTabela.campoAVizualizar* FROM *nomeTabela1* JOIN *nomeTabela2* ON *nomeTabela1.campo* = *nomeTabela2.campo* [WHERE *condição*];

  campos em que os dados são os mesmos (normalmente PK e FK)

  - ex 2 tabelas: select clientes.nome, alergias.alergia from alergias join clientes on alergias.id_cliente=clientes.id;
  - ex 3 tabelas (com as): select livros.titulo as titulo_livro, autores.nome as autor, categorias.nome as categoria from livros join autores on livros.id_autor=autores.id_autor join categorias on livros.id_categoria=categorias.id_categoria;

▷ LEFT JOIN – retorna todos os registos da 1ª tabela e os existentes na 2ª (onde não se cruzarem aparecerá NULL)

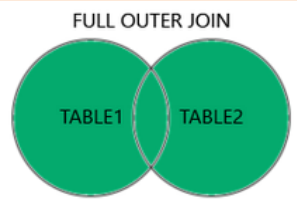
▷ RIGHT JOIN – retorna todos os registos da 2ª tabela e os existentes na 1ª (onde não se cruzarem aparecerá NULL)



FULL JOIN (ou FULL OUTER JOIN) - retorna todos os registos da 1ª tabela e da 2ª (onde não se cruzarem aparecerá NULL)

MySQL, SQLite – usar UNION/UNION ALL de LEFT JOIN e RIGHT JOIN

MS Access – apenas INNER, LEFT, RIGHT



▷ UNION (combina resultados de 2 ou mais select, removendo duplicados)

- SELECT *campo* FROM *nomeTabela1* UNION SELECT *campo* FROM *nomeTabela2*;
  - ex: select cidade from clientes union select cidade from fornecedores;

  ↳ todos os select devem ter o mesmo nº de colunas e as colunas devem ter tipos de dados semelhantes

▷ UNION ALL (como o UNION mas mantém duplicados)

▷ UNION ALL como FULL JOIN

- SELECT *nomeTabela.campoAVisualizar*, *nomeTabela.campoAVizualizar* FROM *nomeTabela1* LEFT JOIN *nomeTabela2* ON *nomeTabela1.campo* = *nomeTabela2.campo* [WHERE *condição*] UNION ALL SELECT

nomeTabela.campoAVisualizar, *nomeTabela.campoAVizualizar* FROM *nomeTabela1* RIGHT JOIN *nomeTabela2* ON *nomeTabela1.campo* = *nomeTabela2.campo* [WHERE *condição*]

- vistas (para guardar consultas usadas frequentemente; pode ser usado com join; atua como uma tabela)
  - criar vista
    - CREATE VIEW *nomeVista* AS SELECT *campo1*, *campo2*, … FROM *nomeTabela* WHERE *condição*;
      - ex: create view [clientes brasileiros] as select nome_cliente, contacto from clientes where pais='brasil';
  - consultar vista
    - SELECT * FROM *nomeVista*;
      - ex: select * from [clientes brasileiros];
  - editar vista
    - CREATE OR REPLACE VIEW *nomeVista* AS SELECT *campo1*, *campo2*, … FROM *nomeTabela* WHERE *condição*;
      - ex: create view [clientes brasileiros] as select nome_cliente, contacto, cidade from clientes where pais='brasil';
  - remover vista
    - DROP VIEW *nomeVista*;
      - ex: drop view[clientes brasileiros];

- procedimentos (quando se usa um conjunto de comandos / uma consulta repetidamente podem ser guardados como procedimento e depois é só chamar o procediemnto)
  - criar:
    - sem parâmetros:
      - DELIMITER // CREATE PROCEDURE nomeProcedimento() BEGIN comandosSQL; END // DELIMITER
        - ex: delimiter // create procedure todos_clientes() begin select * from clientes; END // delimiter;
    - procedimento com 1 parâmetro
      - DELIMITER // CREATE PROCEDURE nomeProcedimento(IN *parâmetro tipoParâmetro*) BEGIN comandosSQL; END // DELIMITER
        - ex: delimiter // create procedure todos_clientes(in cidade varchar(30)) begin select * from clientes where cidade=cidade; end // delimiter;
    - procedimento com 2 parâmetros
      - DELIMITER // CREATE PROCEDURE nomeProcedimento(IN *parâmetro1 tipoParâmetro*, IN *parâmetro2 tipoParâmetro*) BEGIN comandosSQL; END // DELIMITER
        - ex: delimiter // create procedure todos_clientes(in cidade varchar(30), código_postal varchar(8)) begin select * from clientes where cidade=cidade and código_postal=código_postal; end // delimiter;
  - executar:
    - CALL *nomeProcedimento()*;
      - ex: call todos_clientes();
      - ex: call todos_clientes('Lisboa');
      - ex: call todos_clientes('Lisboa', '1000-222');

SQL Server

▷ criar:
- CREATE PROCEDURE nomeProcedimento AS comandosSQL GO;
  - ex: create procedure todos_clientes as select * from customers go;

▷ executar:
- EXEC *nomeProcedimento*;
  - ex: exec todos_clientes;

▷ procedimento com 1 parâmetro
- ex:
  - create procedure todos_clientes @cidade varchar(30) as select * from clientes where city=@city go;
  - exec todos_clientes @city='Lisboa';

▷ procedimento com vários parâmetros
- ex:
  - create procedure todos_clientes @cidade varchar(30), @cosdigo_postal varchar(8) as select * from clientes where city=@city and código_postal=@codigo_postal go;
  - exec todos_clientes @city='Lisboa', @codigo_postal='1000-222';

Complementos de Comandos

▶ com base em um mês de um dado DATE
  ▷ MONTH(*campo*)
    - ex: select * from utilizadores where month(data_registo)=7;

▶ GETDATE()
  ▷ Retorna data e hora atuais (pode ser usado na criação de tabelas ou inserção em tabelas, num campo que careça do dia/hora atual, ou em consultas em que queremos comparar com o dia/hora atual)
    - ex: create table pedidos (id int nut null, pedido int not null, data_pedido date default getdate());
    - ex: insert into pedidos (id, pedido, data_registo) values (10, 59841, getdate());
    - ex: select * from tarefas where prazo < getdate();

▶ AS – para um dado campo aparecer com outro nome naquela consulta (para nomes com espaços devem ser usadas plicas)
  ▷ exs:
    - select livros.titulo as titulo_livro, autores.nome as autor, categorias.nome as categoria from livros join autores on livros.id_autor=autores.id.autor join categorias on livros.id_categoria=categorias.id_categoria;
    - select max(nota) as maior_nota from notas;
  ▷ ↳ na verdade o AS é opcional:
    - max(nota) as maior_nota from notas;
      - =
    - max(nota) maior_nota from notas;

▶ DISTINCT – para devolover os registos distintos (sem mostrar as duplicações)
  ▷ ex: select distinct país from clientes;
  ▷ ex: select count(distinct país) from clientes;

▶ WHERE – usado para filtar resultados, estipulando condições
  ▷ ex: select * from livros where autor='José Saramago';

▶ HAVING (para estipular condições com funções de agregação, que não aceitam WHERE)
    - ex: select count(id_cliente), país from clientes group by país havimg count(id_cliente) > 5;
    - ex: select empregados.apelido, count(pedidos.id_pedido) as quantidade_pedidos from pedidos join empregados on pedidos.id_empregado=empregados.id_empregado group by apelido having count(pedidos.id_pedido) > 10;

▶ EXISTS (para testar a existência de registos numa subconsulta; retorna true/false; oposto NOT EXISTS)
    - ex: listar os fornecedores com produtos com preço menor que 20
      - ex: select nome_fornecedor from fornecedores where exists (select nome_produto where produtos.id_fornecedor=fornecedores.id_fornecedor and preço<20);
    - ex: atualizar apenas preços de produtos em promoção
      - ex: update produtos set preço=preço*0.9 where exists (select * from descontos where descontos.id_produto=produtos.id_produto);

- AND – para estipular várias condições que devem ser verdadeiras
  - ex: select * from clientes where país='Espanha' and nome like 'G%';
- OR – para estipular que pelo menos uma das condições que deve ser verdadeira
  - ex: select * from clientes where país='Espanha' ou país='Portugal';
- AND e OR – podem ser usados no mesmo comando mas podem ser necessários parêntesis para especificar
  - ex: select * from clientes where país='Espanha' and (nome like 'G%' or nome like 'R%');
    - ↳ devolve todos os clientes espanhóis cujo nome começa por G ou R
  - ex: select * from clientes where país='Espanha' and nome like 'G%' or nome like 'R%';
    - ↳ devolve todos os clientes que sejam espanhóis e cujo nome começa por G ou todos os clientes cujo nome comece por R
- NOT – para estipular o oposto do operador (quase como diferente de)
  - ex: select * from clientes where not país='Espanha';
  - ex: select * from clientes where nome not like 'A%';

- IS NULL / IS NOT NULL – identifica células vazias / não vazias
  - ex: select nome, telemóvel, morada from clientes where morada is null;

- CONCAT (concatenar colunas)
  - ex: select nome_cliente, concat(morada,', ',codiigo_postal,' ',cidade) as morada;

- GROUP BY (agrupa linhas que têm os mesmos valores numa linha sumário; normalmente usado com funções de agregação)
  - ex: contar quantos clientes por país
    - select count(id_cliente), país from clientes group by país;

- ANY / SOME (realiza comparação entre um valor de uma coluna e um conjunto de outros valores; retorna true se **algum** valor se enquadrar na condição)
  - SELECT *campo* FROM *nomeTabela* WHERE *campo* *operador** ANY (SELECT *campo* FROM *nomeTabela* WHERE *condição*); (*operadores: =, <>, !=, >, >=, <, <=)
    - ex: listar os produtos dos quais foram feitos pedidos de 10 unidades
      - ex: select nome_produto from produtos where id_produto = any (select id_produto from pedidos where quantidade = 10);
    - ex: listar clientes que fizeram mais compras que qualquer cliente VIP
      - ex: select nome_cliente from clientes where total_pedidos > any (select total_pedidos from clientes where vip = true);
- ALL (como o ANY mas retorna true se **todos** os valores se enquadrarem na condição
  - SELECT ALL *campo* FROM *nomeTabela* WHERE *condição*;
    - raro dado que ALL já é o comportamento padrão do select
  - SELECT *campo* FROM *nomeTabela* WHERE *campo* *operador** ALL (SELECT *campo* *nomeTabela* WHERE *condição*); (*operadores: =, <>, !=, >, >=, <, <=)
    - ex: listar clientes que fizeram mais compras que todos os clientes VIP
      - ex: select nome_cliente from clientes where total_pedidos > all (select total_pedidos from clientes where vip = true);

Operador	Com ANY	Com ALL
>	Maior que **algum** valor	Maior que **todos** os valores
<	Menor que **algum** valor	Menor que **todos** os valores
=	Igual a **algum** valor	Igual a **todos** os valores (raro e difícil)

▶ CASE (percorre as condições e retorna o valor da 1ª condição satisfeita)
- ▷ CASE WHEN *condição1* THEN *resultado1* WHEN *condição2* THEN *resultado2* WHEN *condiçãoN* THEN *resultadoN* ELSE *resultado* END;
  - ex: indicar para cada preço se é barato, caro ou médio, numa nova coluna
    - ex: select nome_produto, preço, case when preço<10 then 'barato' when preço between 10 and 50 then 'médio' else 'caro' end as faixa_de_preco from produtos;

▶ funções NULL
- ▷ IFNULL() (devolve um resultado para operações com valores nulos; suporta 2 argumentos)
  - ex: queremos calcular o valor total das unidades em stock e das unidades pedidas, mas estas ultimas podem conter valores nulos o que daria um resultado nulo
    - select nome_produto, preço_unidade * (unidades_stock + ifnull(unidades_pedidas, 0)) from produtos;
- ▷ COALESCE() (como IFNULL() mas suporta 2 ou mais argumentos e retorna o 1º não nulo)
  - ex: select nome_produto, preço_unidade * (unidades_stock + coalesce(unidades_pedidas, 0)) from produtos;
  - ex: select nome, coalesce(telefone, email, 'sem contacto') as contacto_preferencial from clientes;

SQL Server
- ▷ ISNULL() (devolve um valor alternativo quando uma expressão é nula)
  - ex: select nome_produto, preço_unidade * (unidades_stock + isnull(unidades_pedidas, 0)) from produtos;
- ▷ COALESCE() – igual

MS Acess
- ▷ ISNULL() (devolve true (-1) quando uma expressão é nula, senão devolve false (0))
  - ex: select nome_produto, preço_unidade * (unidades_stock + iif(isnull(unidades_pedidas), 0, unidades_pedidas)) from produtos;

Oracle
- ▷ NVL() (devolve true (-1) quando uma expressão é nula, senão devolve false (0))
  - ex: select nome_produto, preço_unidade * (unidades_stock + nvl(unidades_pedidas, 0)) from produtos;
- ▷ COALESCE() – igual

▶ comentários
- ▷ – comentário
  - ex: select * from clientes; *-- selecionar todos*
- ▷ # comentário
  - ex: select * from clientes; *# selecionar todos*
- ▷ /* comentário */
  - ex: */*select * from clientes;*
    *select * from produtos;*
    *select * from pedidos;*
    *select * from categorias;*/*
    select * from fornecedores;
  - ex: select nome_cliente, */*cidade,*/* pais from clientes;

Tabelas Sumárias

▶ Tipos de dados

▷ Texto (String)

Tipo de Dado	Descrição
CHAR(tamanho)	Uma string de comprimento FIXO (pode conter letras, números e caracteres especiais). O parâmetro tamanho especifica o comprimento da coluna em caracteres - pode ser de 0 a 255. O predefinido é 1.
VARCHAR(tamanho)	Uma string de comprimento VARIÁVEL (pode conter letras, números e caracteres especiais). O parâmetro tamanho especifica o comprimento máximo da string em caracteres. Pode ser de 0 a 65535 bytes.
BINARY(tamanho)	Igual a CHAR(), mas armazena strings de bytes binários. O parâmetro tamanho especifica o comprimento da coluna em bytes. O predefinido é 1.
VARBINARY(tamanho)	Igual a VARCHAR(), mas armazena strings de bytes binários. O parâmetro tamanho especifica o comprimento máximo da coluna em bytes.
TINYBLOB	Para BLOBs (Binary Large Objects). Tamanho máximo: 255 bytes
TINYTEXT	Contém uma string com um comprimento máximo de 255 caracteres
TEXT	Contém uma string com um comprimento máximo de 65.535 bytes
BLOB	Para BLOBs (Binary Large Objects). Armazena até 65.535 bytes de dados
MEDIUMTEXT	Contém uma string com um comprimento máximo de 16.777.215 caracteres
MEDIUMBLOB	Para BLOBs (Binary Large Objects). Armazena até 16.777.215 bytes de dados
LONGTEXT	Contém uma string com um comprimento máximo de 4.294.967.295 caracteres
LONGBLOB	Para BLOBs (Binary Large Objects). Armazena até 4.294.967.295 bytes de dados.
ENUM(val1, val2, val3, ...)	Um objeto string que pode ter apenas um valor, escolhido de uma lista de valores possíveis. Pode listar até 65.535 valores numa lista ENUM. Se for introduzido um valor que não esteja na lista, será introduzido um valor em branco. Os valores são ordenados pela ordem em que os introduz.
SET(val1, val2, val3, ...)	Um objeto string que pode ter 0 ou mais valores, escolhidos de uma lista de valores possíveis. Pode listar até 64 valores numa lista SET.

▷ Numéricos

Tipo de Dado	Descrição
BIT(*tamanho*)	Um tipo de valor de bit. O número de bits por valor é especificado em tamanho. O parâmetro tamanho pode conter um valor de 1 a 64. O valor predefinido para tamanho é 1.
TINYINT(*tamanho*)	Um inteiro muito pequeno. O intervalo com sinal vai de -128 a 127. O intervalo sem sinal vai de 0 a 255. O parâmetro de tamanho especifica a largura máxima de visualização (que é 255).
BOOL	O zero é considerado falso, os valores diferentes de zero são considerados verdadeiros.
BOOLEAN	Igual ao BOOL
SMALLINT(*tamanho*)	Um inteiro pequeno. O intervalo SIGNED vai de -32768 a 32767. O intervalo UNSIGNED vai de 0 a 65535. O parâmetro de tamanho especifica a largura máxima de visualização (que é 255).
MEDIUMINT(*tamanho*)	Um número inteiro médio. O intervalo SIGNED vai de -8388608 a 8388607. O intervalo UNSIGNED vai de 0 a 16777215. O parâmetro tamanho especifica a largura máxima de visualização (que é 255).
INT	Um número inteiro médio. O intervalo SIGNED vai de -2147483648 a 2147483647. O intervalo UNSIGNED vai de 0 a 4294967295. O parâmetro size especifica a largura máxima de visualização (que é 255).
INTEGER	Igual a INT
BIGINT(*tamanho*)	Um inteiro grande. O intervalo SIGNED vai de -9223372036854775808 a 9223372036854775807. O intervalo UNSIGNED vai de 0 a 18446744073709551615. O parâmetro size especifica a largura máxima de visualização (que é 255).
FLOAT(*p*)	Um número de ponto flutuante. O MySQL utiliza o valor p para determinar se o FLOAT ou o DOUBLE deve ser utilizado para o tipo de dados resultante. Se p for de 0 a 24, o tipo de dados passa a ser FLOAT(). Se p for de 25 a 53, o tipo de dados passa a ser DOUBLE().
DOUBLE(*tamanho*, *d*)	Um número de ponto flutuante de tamanho normal. O número total de dígitos é especificado em tamanho. O número de dígitos após a vírgula é especificado no parâmetro d.
DOUBLE PRECISION(*tamanho*, *d*)	
DECIMAL(*tamanho*, *d*)	Um número exato de ponto fixo. O número total de dígitos é especificado em tamanho. O número de dígitos após a vírgula é especificado no parâmetro d. O número máximo para tamanho é 65. O número máximo para d é 30. O valor predefinido para tamanho é 10. O valor predefinido para d é 0.
DEC(*tamanho*, *d*)	Igual a DECIMAL(tamanho,d)

▷ Data e Hora

Tipo de Dado	Descrição
DATE	Uma data. Formato: AAAA-DD-MM. O intervalo suportado é de '1000-01-01' a '9999-12-31'
DATETIME(*fsp*)	Uma combinação de data e hora. Formato: AAAA-MM-DD hh:mm:ss. O intervalo suportado é de '1000-01-01 00:00:00' a '9999-12-31 23:59:59'. Adicionar DEFAULT e ON UPDATE na definição da coluna para obter a inicialização e a atualização automáticas para a data e hora atuais.
TIMESTAMP(*fsp*)	Um carimbo de data/hora. Os valores de TIMESTAMP são armazenados como o número de segundos desde a era Unix ('1970-01-01 00:00:00' UTC). Formato: AAAA-MM-DD hh:mm:ss. O intervalo suportado é de '1970-01-01 00:00:01' UTC a '2038-01-09 03:14:07' UTC. A inicialização e atualização automáticas para a data e hora atuais podem ser especificadas utilizando DEFAULT CURRENT_TIMESTAMP e ON UPDATE CURRENT_TIMESTAMP na definição da coluna.
TIME(*fsp*)	Uma hora. Formato: hh:mm:ss. O intervalo suportado é de '-838:59:59' a '838:59:59'.
YEAR	Um ano no formato de quatro dígitos. Valores permitidos no formato de quatro dígitos: 1901 a 2155 e 0000.

*fsp (fractional seconds precision) – quando não especificado é 0 por defeito.

fsp	Precisão de segundos	Exemplo de valor armazenado
0	Sem frações	2025-08-13 18:45:00
1	Décimos	2025-08-13 18:45:00.1
2	Centésimos	2025-08-13 18:45:00.12
3	Milissegundos	2025-08-13 18:45:00.123
4	Décimos de milissegundo	2025-08-13 18:45:00.1234
5	Centésimos de milissegundo	2025-08-13 18:45:00.12345
6	Microssegundos	2025-08-13 18:45:00.123456

▶ Palavras-chave

Palavra-chave	Descrição	Observações
ADD	Adiciona uma coluna a uma tabela existente	Usado com ALTER TABLE
ADD CONSTRAINT	Adiciona uma restrição após a criação da tabela	Pode ser PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK
ALL	Retorna verdadeiro se todos os valores da subconsulta atenderem à condição	Usado com operadores de comparação
ALTER	Modifica estrutura da tabela: adiciona, remove ou altera colunas	Comando DDL
ALTER COLUMN	Altera o tipo de dados ou atributos de uma coluna	Usado dentro de ALTER TABLE
ALTER TABLE	Modifica a estrutura de uma tabela	Totalmente suportado
AND	Retorna verdadeiro se ambas condições forem verdadeiras	Operador lógico
ANY	Retorna verdadeiro se qualquer valor da subconsulta atender à condição	Similar ao IN, mas com operadores
AS	Renomeia uma coluna ou tabela com um alias	Usado em SELECT, JOIN, CREATE VIEW
ASC	Ordena resultados em ordem crescente	Padrão em ORDER BY
BETWEEN	Seleciona valores dentro de um intervalo	Inclusivo: inclui os limites
CASE	Cria saídas condicionais	Equivalente a IF...THEN...ELSE
CHECK	Restrição que limita valores de uma coluna	Suporte completo desde MySQL 8.0.16
COLUMN	Usado em ALTER TABLE para modificar ou excluir colunas	Opcional: ALTER TABLE ... MODIFY COLUMN ...
CONSTRAINT	Adiciona ou remove restrições	Inclui PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK
CREATE	Cria banco de dados, tabela, índice, view ou procedimento	Comando DDL
CREATE DATABASE	Cria um novo banco de dados	Totalmente suportado
CREATE INDEX	Cria um índice (permite valores duplicados)	Usa CREATE INDEX ... ON tabela(coluna)
CREATE OR REPLACE VIEW	Funciona normalmente para atualizar views	
CREATE TABLE	Cria uma nova tabela	Comando essencial
CREATE PROCEDURE	Cria uma stored procedure	Requer DELIMITER para definir blocos
CREATE UNIQUE INDEX	Cria índice único (sem valores duplicados)	Garante unicidade
CREATE VIEW	Cria uma view baseada em SELECT	Suporte completo
DATABASE	Refere-se à criação ou exclusão de banco de dados	Usado com CREATE ou DROP
DEFAULT	Define valor padrão para uma coluna	Pode ser alterado com ALTER TABLE
DELETE	Exclui linhas de uma tabela	Usado com WHERE
DESC	Ordena o conjunto de resultados em ordem decrescente	Usado com ORDER BY coluna DESC

DISTINCT	Seleciona apenas valores distintos (diferentes)	Funciona com SELECT DISTINCT coluna
DROP	Exclui coluna, restrição, base de dados, índice, tabela ou visão	Cada tipo requer sintaxe específica (DROP TABLE, DROP INDEX, etc.)
DROP COLUMN	Exclui uma coluna de uma tabela	Usado com ALTER TABLE tabela DROP COLUMN coluna
DROP CONSTRAINT	Exclui restrições como UNIQUE, PRIMARY KEY, FOREIGN KEY ou CHECK	
DROP DATABASE	Exclui uma base de dados existente	DROP DATABASE nome
DROP INDEX	Exclui um índice de uma tabela	DROP INDEX nome ON tabela
DROP TABLE	Exclui uma tabela existente	DROP TABLE nome
DROP VIEW	Exclui uma visão (view)	DROP VIEW nome
EXISTS	Testa a existência de registros em uma subconsulta	Usado com WHERE EXISTS (subconsulta)
FOREIGN KEY	Chave estrangeira usada para relacionar duas tabelas	Definida com FOREIGN KEY (coluna) REFERENCES tabela(coluna)
FROM	Especifica de qual tabela selecionar ou excluir dados	Parte essencial do SELECT e DELETE
GROUP BY	Agrupa resultados (usado com funções agregadas)	GROUP BY coluna
HAVING	Usado em vez de WHERE com funções agregadas	HAVING COUNT(*) > 1
IN	Permite especificar múltiplos valores em uma cláusula WHERE	WHERE coluna IN (valor1, valor2, ...)
INDEX	Cria ou exclui um índice	CREATE INDEX, DROP INDEX
INNER JOIN	Retorna linhas com valores correspondentes em ambas as tabelas	SELECT ... FROM tabela1 INNER JOIN tabela2 ON condição
INSERT INTO	Insere novas linhas em uma tabela	INSERT INTO tabela (colunas) VALUES (valores)
INSERT INTO SELECT	Copia dados de uma tabela para outra	INSERT INTO tabela2 SELECT ... FROM tabela1
IS NULL	Testa valores nulos	WHERE coluna IS NULL
IS NOT NULL	Testa valores não nulos	WHERE coluna IS NOT NULL
JOIN	Junta tabelas	Pode ser INNER, LEFT, RIGHT, etc.
LEFT JOIN	Retorna todas as linhas da tabela da esquerda e correspondentes da direita	LEFT JOIN
LIKE	Pesquisa por padrão em uma coluna	WHERE coluna LIKE 'A%'
LIMIT	Especifica o número de registros a retornar	LIMIT 10
NOT	Inclui linhas onde a condição não é verdadeira	WHERE NOT condição
NOT NULL	Restrição que impede valores nulos	coluna tipo NOT NULL
OR	Inclui linhas onde qualquer condição é verdadeira	WHERE condição1 OR condição2
ORDER BY	Ordena os resultados em ordem crescente ou decrescente	ORDER BY coluna ASC|DESC
PRIMARY KEY	Identifica exclusivamente cada registro na tabela	PRIMARY KEY (coluna)

PROCEDURE	Uma stored procedure	Criada com CREATE PROCEDURE, chamada com CALL
RIGHT JOIN	Retorna todas as linhas da tabela da direita e correspondentes da esquerda	RIGHT JOIN
SELECT	Seleciona dados de uma base de dados	SELECT coluna FROM tabela
SELECT DISTINCT	Seleciona apenas valores distintos	SELECT DISTINCT coluna
SET	Especifica colunas e valores a serem atualizados	Usado com UPDATE tabela SET coluna = valor
TABLE	Cria, modifica ou exclui tabelas e colunas	CREATE TABLE, ALTER TABLE, DROP TABLE
TRUNCATE TABLE	Exclui os dados de uma tabela, mas não a tabela	TRUNCATE TABLE nome
UNION	Combina os resultados de duas ou mais instruções SELECT (sem duplicados)	SELECT ... UNION SELECT ...
UNION ALL	Combina os resultados de duas ou mais instruções SELECT (permite duplicados)	SELECT ... UNION ALL SELECT ...
UNIQUE	Restrição que garante que todos os valores em uma coluna sejam únicos	Usado com CREATE TABLE ou ALTER TABLE
UPDATE	Atualiza linhas existentes em uma tabela	UPDATE tabela SET coluna = valor WHERE condição
VALUES	Especifica os valores em uma instrução INSERT INTO	INSERT INTO tabela VALUES (...)
VIEW	Cria, atualiza ou exclui uma visão (view)	CREATE VIEW, ALTER VIEW, DROP VIEW
WHERE	Filtra os resultados para incluir apenas registros que satisfazem uma condição	Usado com SELECT, UPDATE, DELETE

▶ Funções MySQL

▷ Funções de String

Função	Descrição	Sintaxe	Exemplo
ASCII	Retorna o valor ASCII do primeiro caractere	ASCII(*character*)	select ascii(titulo) from livros;
CHAR_LENGTH	Retorna o comprimento da string em caracteres. Igual a CHARACTER_LENGTH	CHAR_LENGTH(*string*)	select char_length(nome_cliente) from clientes;
CHARACTER_LENGTH	Retorna o comprimento da string em caracteres. Igual a CHAR_LENGTH	CHARACTER_LENGTH(*string*)	select character_length(nome_cliente) from clientes;
CONCAT	Concatena duas ou mais expressões	CONCAT(*expressão1*, *expressão2*, …)	select concat(morada, ', ', código_postal, ' ', cidade) as morada from clientes;
CONCAT_WS	Concatena expressões com um separador	CONCAT_WS(separador, expressão1, *expressão2*, …)	select concat(' ', morada, código_postal, cidade) as morada from clientes;
FIELD	Retorna a posição de um valor dentro de uma lista	FIELD(*valorAAnalisar*, *val1*, *val2*, …)	select field('q', 's', 'q', 'l'); →retorna 2
FIND_IN_SET	Retorna a posição de uma string dentro de uma lista separada por vírgulas	FIND_IN_SET(*string*, *lista*)	select find_in_set('q', 's,q,l); →retorna 2
FORMAT	Formata um número com separadores e casas decimais	FORMAT(*número*, *decimais*)	select format(250500.5634, 2) →retorna 250,500.56

INSERT	Insere uma string dentro de outra, em uma posição específica	INSERT(*string*, *posiçãoInicial*, *nºcaracteres*, *string2*)	select insert('Protugal', 2, 2, 'or');
			
			select inset(nome), 1, 0, 'Sra. ') from clientes;

INSTR	Retorna a posição da primeira ocorrência de uma string dentro de outra	INSTR(*string1*, *substring*):	select instr('Sandra', 'a'); →retorna 2
LCASE	Converte a string para minúsculas. Igual a LOWER	LCASE(*text*):	select lcase(nome_cliente) from clientes;
LEFT	Extrai um número de caracteres a partir da esquerda. Ver RIGHT	LEFT(*string*, *nºcaracteres*);	select left('Sandra Mota', 6); →retorna Sandra

LENGTH	Retorna o comprimento da string em bytes	LENGTH(*string*)	Deve retornar o nº de bytes, mas dependendo da configuração do sistema pode retornar caractares
LOCATE	Retorna a posição da primeira ocorrência de uma substring. Semelhante a **POSITION**	LOCATE(*substring*, *string*, *inicio*) Inicio é opcional e por defeito é 1	select locate('a', 'Sandra'); →retorna 2 select locate('a', 'Sandra', 3); →retorna 6 select locate('a', nome_cliente) from clientes;
LOWER	Converte a string para minúsculas. Igual a **LCASE**	LOWER(*text*):	select lower(nome_cliente) from clientes;
LPAD	Preenche a string à esquerda com outra string até atingir determinado comprimento. Ver **RPAD**	LPAD(*string1*, *tamanho*, *string2*)	select lpad('12545', 8, '0'); →retorna 00012345 select lpad(encomendas, 10, 'XXX');
LTRIM	Remove espaços à esquerda da string. Ver **RTRIM** e **TRIM**	LTRIM(*string*)	select ltrim(' Sandra'); →retorna Sandra select ltrim(nome_cliente) from clientes;
MID	Extrai uma substring a partir de uma posição específica. Igual a **SUBSTRING** e **SUBSTR**	MID(*string*, *inicio*, *tamanho*) *inicio* - podemos contar do incio (1º carácter – posição1) ou do fim (último carácter – posição -1) *tamanho* – opcional, se não indicado é dado o string até ao fim	select mid('Sandra', 3, 2); →retorna nd select mid('Sandra', -3, 2); →retorna dr select mid('Sandra', 3); →retorna ndra
POSITION	Retorna a posição da primeira ocorrência de uma substring. Semelhante a **LOCATE**	POSITION(*substring* IN *string*)	select position('a' IN 'Sandra'); →retorna 2 select position('a' IN nome_cliente) from clientes;
REPEAT	Repete uma string o número de vezes especificado	REPEAT(*string*, *nºvezes*)	select repeat('ABC', 3) →retorna ABCABCABC

REPLACE	Substitui todas as ocorrências de uma substring por outra	REPLACE(*string*, *textoAntigo*, *textoNovo*)	select repeat('Mª Francisca', 'Mª', 'Maria');
REVERSE	Inverte os caracteres de uma string	REVERSE(*string*)	select reverse('ABC'); →retorna CBA
RIGHT	Extrai um número de caracteres a partir da direita. Ver **LEFT**	RIGHT(*string*, *nºcaracteres*);	select right('Sandra Mota', 4); →retorna Mota
RPAD	Preenche a string à direita com outra string até atingir determinado comprimento. Ver **LPAD**	RPAD(*string1*, *tamanho*, *string2*)	select rpad('12545', 8, '0'); →retorna 12345000 select rpad(encomendas, 10, 'XXX');
RTRIM	Remove espaços à direita da string. Ver **LTRIM** e **TRIM**	RTRIM(*string*)	select rtrim('Sandra '); →retorna Sandra select rtrim(nome_cliente) from clientes;
SPACE	Retorna uma string com o número especificado de espaços em branco	SPACE(*número*)	select space(5); →retorna ' ' select concat('Sandra', space(3), 'Mota'); →retorna 'Sandra Mota'
STRCMP	Compara duas strings	STRCMP(*string1*, *string2*) Tem em conta o valor ASCII de cada caractere por ordem de aparecimento Se string1 = string2, retorna 0 Se string1 < string2, retorna -1 Se string1 > string2, retorna 1	select strcmp('Sandra', 'Sandra'); →retorna 0 select strcmp('Ana', 'Sandra'); →retorna -1 select strcmp('Sandra', 'Ana'); →retorna 1

SUBSTR	Extrai uma substring a partir de uma posição específica. Igual a **SUBSTRING** e **MID**	SUBSTR(*string*, *inicio*, *tamanho*) *inicio* - podemos contar do incio (1º carácter – posição1) ou do fim (último carácter – posição -1) *tamanho* – opcional, se não indicado é dado o string até ao fim	select substr('Sandra', 3, 2); →retorna nd select substr('Sandra', -3, 2); →retorna dr select substr('Sandra', 3); →retorna ndra
SUBSTRING	Extrai uma substring a partir de uma posição específica. Igual a **MID** e **SUBSTR**	SUBSTRING(*string*, *inicio*, *tamanho*) *inicio* - podemos contar do incio (1º carácter – posição1) ou do fim (último carácter – posição -1) *tamanho* – opcional, se não indicado é dado o string até ao fim	select substring('Sandra', 3, 2); →retorna nd select substring ('Sandra', -3, 2); →retorna dr select substring ('Sandra', 3); →retorna ndra
SUBSTRING_INDEX	Retorna parte da string antes ou depois de um delimitador	SUBSTRING_INDEX(*string*, *delimitador*, *nºvezes*) *nºvezes* – se for positivo retorna parte antes do delimitador, se negativo rertona a parte após	select substring_index('Apontamentos.Sandra.Mota', '.', 1); →retorna Apontamentos select substring_index('Apontamentos.Sandra.Mota', '.', 2); →retorna Apontamentos.Sandra select substring_index('Apontamentos.Sandra.Mota', '.', -1); →retorna Mota

TRIM	Remove espaços à esquerda e à direita da string. Ver **LTRIM** e **RTRIM**	TRIM(*string*)	select trim(' Sandra '); →retorna Sandra select trim(nome_cliente) from clientes;
UCASE	Converte a string para maiúsculas. Igual a **UPPER**	UCASE(*texto*)	select ucase('Sandra'); →retorna SANDRA select ucase(nome_cliente) from clienets;
UPPER	Converte a string para maiúsculas. Igual a **UCASE**	UPPER(*texto*)	select upper('Sandra'); →retorna SANDRA select upper(nome_cliente) from clienets;

▷ Funções Numéricas

Função	Descrição	Sintaxe	Exemplo
ABS	Retorna o valor absoluto de um número	ABS(*número*)	select abs(-5); → retorna 5
ACOS	Retorna o arco cosseno de um número	ACOS(*número*)	select acos(0.25); → retorna 1.318116071652818
ASIN	Retorna o arco seno de um número	ASIN(*número*)	select asin(0.25); → retorna 0.25268025514207865
ATAN	Retorna o arco tangente de um ou dois números	ATAN(*número*) ou ATAN2(*a*, *b*) a, b – 2 valores para calcular arco tangente	select atan(2.5); → retorna 1.1902899496825317 select atan(-0.8, 2); → retorna -0.3805063771123649
ATAN2	Retorna o arco tangente de dois números	ATAN2(*a*, *b*) a, b – 2 valores para calcular arco tangente	select atan2(-0.8, 2); → retorna -0.3805063771123649
AVG	Retorna o valor médio de uma expressão	AVG(*expressão*)	select avg(preço) from produtos;
CEIL	Retorna o menor número inteiro ≥ ao número informado. Igual a **CEILING.** Ver **ROUND**, **FLOOR** e **TRUNCATE**	CEIL(*número*)	select ceil(4.2); → retorna 5

CEILING	Retorna o menor número inteiro ≥ ao número informado. Igual a **CEIL**	CEILING(*número*)	select ceiling(4.2); → retorna 5
COS	Retorna o cosseno de um número	COS(*número*)	select cos(2); → retorna -0.4161468365471424
COT	Retorna a cotangente de um número	COT(*número*)	select cot(6); → retorna -3.436353004180128
COUNT	Retorna o número de registros retornados por uma consulta	COUNT(*expressão*)	select count(*) from clientes;
DEGREES	Converte um valor em radianos para graus. Ver **RADIANS** e **PI**	DEGREES(*número*)	select degrees(1.5); → retorna 85.94366926962348
DIV	Usado para divisão inteira	*num1* DIV *num2*	select 10 div 3; → retorna 3
EXP	Retorna *e* elevado à potência de um número. Ver **LOG** e **LN**	EXP(*número*)	slelect exp(1); → retorna 2.718281828459045
FLOOR	Retorna o maior número inteiro ≤ ao número informado. Ver **ROUND**, **CEIL** e **TRUNCATE**	FLOOR(*número*)	select floor(4.8); → retorna 4
GREATEST	Retorna o maior valor entre os argumentos fornecidos. Ver **LEAST**	GREATEST(*arg1*, *arg2*, …) Quando usado com texto retorna o maior valor ASCII (ver STRCMP())	select greatest(3, 7, 2); → retorna 7
LEAST	Retorna o menor valor entre os argumentos fornecidos. Ver **GREATEST**	LEAST(*arg1*, *arg2*, …) Quando usado com texto retorna o menor valor ASCII (ver STRCMP())	select least(3, 7, 2); → retorna 2
LN	Retorna o logaritmo natural de um número. Ver **EXP** e **LOG**	LN(*número*)	select ln(2); → retorna 0.6931471805599453
LOG	Retorna o logaritmo natural ou logaritmo de base especificada. Ver **EXP** e **LN**	LOG(*número*) ou LOG(*base*, *número*) *número* – deve ser maior que 0 *base* – deve ser maior que 1	select log(2); → retorna 0.6931471805599453 select log(2, 4); → retorna 2
LOG10	Retorna o logaritmo de base 10	LOG10(*número*)	selet log10(100) → retorna 2

LOG2	Retorna o logaritmo de base 2	LOG2(*número*)	select log2(8) → retorna 3
MAX	Retorna o maior valor em um conjunto	MAX(*expressão*)	select max(salario) from empregados;
MIN	Retorna o menor valor em um conjunto	MIN(*expressão*)	select min(salario) from empregados;
MOD	Retorna o resto da divisão entre dois números	MOD(*num1*, *num2*) ou *num1* MOD *num2* ou *num1* % *num2* *num1* – a dividir por num2 *num2* – divisor	select mod(18, 4); → retorna 2
PI	Retorna o valor de π. Ver **DEGREES** e **RADIANS**	PI()	select pi(); → retorna 3.141593
POW	Retorna um número elevado à potência de outro número. Igual a **POWER**	POW(*num1*, *num2*) *num1* – base *num2* – exponente	select pow(2, 3); → retorna 8
POWER	Retorna um número elevado à potência de outro número. Igual a **POW**	POWER(*num1*, *num2*) *num1* – base *num2* – exponente	select power(2, 3) ; → retorna 8
RADIANS	Converte um valor em graus para radianos. Ver **DEGREES** e **PI**	RADIANS(*número*)	select radians(180); → retorna 3.141592653589793 (pi())

RAND	Retorna um número decimal aleatório >= 0 e <1	RAND(*seed*)
		
		seed – opcional; se especificado retorna sempre o mesmo número aleatório quando usada a mesma seed
			select rand();
			
			select rand(6);
			
			select rand()*(10-5)+5; → retorna um nº decimal aleatório >=5 e <10
			
			select floor(rand()*(10-5+1)+5); → retorna um nº decimal aleatório >=5 e <=10

ROUND	Arredonda um número para o número de casas decimais especificado. Ver **FLOOR**, **CEIL** e **TRUNCATE**	ROUND(*número*, *decimais*)	select round(4.567, 2); → retorna 4.57
SIGN	Retorna o sinal de um número: -1, 0 ou 1	SIGN(*número*) Se número > 0, retorna 1 Se número = 0, retorna 0 Se número < 0, retorna -1	select sign(-10); → -1
SIN	Retorna o seno de um número	SIN(*número*)	select sin(2); → retorna 0.9092974268256817 select sin(pi()/2); → retorna 1
SQRT	Retorna a raiz quadrada de um número	SQRT(*número*)	select sqrt(16); → retorna 4
SUM	Calcula a soma de um conjunto de valores	SUM(*expressão*)	select sum(quantidade) from pedidos;
TAN	Retorna a tangente de um número	TAN(*número*)	select tan(1.75); → retorna -5.52037992250933 select tan(pi()/4); → retorna 1
TRUNCATE	Trunca um número para o número de casas decimais especificado (sem arredondar). Ver **ROUND**, **FLOOR** e **CEIL**	TRUNCATE(*número*, *decimais*)	select truncate(4.567, 2); → 4.56

▷ Funções de data e hora

Função	Descrição	Sintaxe	Exemplo
ADDDATE	Adiciona um intervalo de tempo/data a uma data. Igual a **DATE_ADD.** Oposto de **DATE_SUB** e **DATESUB**	ADDDATE(*data*, INTERVAL *valor unidade*) ou ADDDATE(*data*, *dias*) Valor: tanto positivo como negativo (positivo acrescenta, negativo retira) Unidades: *	select adddate('2017-06-15', interval 10 day); → retorna 2017-06-25 select adddate('2017-06-15 09:34:21', interval -3 hour); → retorna 2017-06-15 06:34:21
ADDTIME	Adiciona um intervalo de tempo a um tempo/datetime	ADDTIME(*datetime*, *tempo*)	select addtime('2017-06-15 09:34:21', '2'); → adiciona 2seg select addtime('2017-06-15 09:34:21.000001', '2:10:5.000003'); → adiciona 2h, 10min, 5seg e 3microsegundos
CURDATE	Retorna a data atual. Igual a **CURRENT_DATE**	CURDATE() Retorna 'AAAA-MM-DD'(string) ou AAAAMMDD (numérico)	select curdate();→ ex: retorna 2025-08-14 select curdate()+1; → ex: retorna 20250815
CURRENT_DATE	Retorna a data atual. Igual a **CURDATE**	CURRENT_DATE() Retorna 'AAAA-MM-DD'(string) ou AAAAMMDD (numérico)	select current_date();→ ex: retorna 2025-08-14 select current_date()+1; → ex: retorna 20250815
CURRENT_TIME	Retorna a hora atual. Igual a **CURTIME**	CURRENT_TIME() Retorna 'HH-MM-SS'(string) ou HHMMSS.uuuuuu (numérico)	select current_time(); → ex: retorna 16:47:36 select current_time()+1; → ex: retorna 164737

CURRENT_TIMESTAMP	Retorna data e hora atuais	CURRENT_TIMESTAMP() Retorna 'AAAA-MM-DD HH-MM-SS'(string) ou AAAAMMDD HHMMSS.uuuuuu (numérico)	select current_timestamp();→ ex: retorna 2025-08-14 16:51:46 select current_timestamp();→ ex: retorna 20250814165147
CURTIME	Retorna a hora atual. Igual a **CURRENT_TIME**	CURTIME() Retorna 'HH-MM-SS'(string) ou HHMMSS.uuuuuu (numérico)	select curtime(); → ex: retorna 16:47:36 select curtime()+1; → ex: retorna 164737
DATE	Extrai a parte da data de uma expressão datetime	DATE(*datetime*)	select date('2023-01-01 10:30:00'); → retorna 2023-01-01 select date(data_pedido) from pedidos;
DATEDIFF	Retorna o número de dias entre duas datas	DATEDIFF(*data1*, *data2*)	select datediff('2023-01-10', '2023-01-01'); → retorna 9 select datediff('2017-06-25 09:34:21', '2017-06-15 09:25:35'); → retorna 10
DATE_ADD	Adiciona um intervalo de tempo/data a uma data. Igual a **ADDDATE**. Oposto de **DATE_SUB** e **DATESUB**	DATE_ADD(*data*, INTERVAL *valor unidade*) ou DATE_ADD(*data*, *dias*) Valor: tanto positivo como negativo (positivo acrescenta, negativo retira) Unidades: *	select date_add('2017-06-15', interval 10 day); → retorna 2017-06-25 select date_add('2017-06-15 09:34:21', interval -3 hour); → retorna 2017-06-15 06:34:21
DATE_FORMAT	Formata uma data	DATE_FORMAT(*data*, *formato*) Formatos: **	select date_format("2017-06-15", "%Y"); → retorna 2017 select date_format(now(), '%d/%m/%y'); → retorna ex: 14/08/25 select date_format(data_pedidos, '%d/%m/%y') from pedidos;

DATE_SUB	Subtrai um intervalo de tempo/data de uma data. Igual a **DATE_SUB**. Oposto de **DATE_SUB** e **DATESUB** e **ADDDATE**	DATE_SUB(*data*, INTERVAL *valor unidade*) Valor: tanto positivo como negativo (negativo acrescenta, positivo retira) Unidades: *	select date_sub('2023-01-10', interval 5 day); → retorna 2023-01-05 select date_sub('2023-01-10', interval -5 day); → retorna 2023-01-15
DAY	Retorna o dia do mês de uma data. Igual a **DAYOFMONTH**	DAY(*data*)	select day('2023-01-15'); → retorna 15 select day(curdate));
DAYNAME	Retorna o nome do dia da semana	DAYNAME(*data*) Sunday,Monday,Tuesday, Wednesday,Thursday,Friday, Saturday	select dayname('2023-01-15'); → retorna Sunday select dayname(curdate());
DAYOFMONTH	Retorna o dia do mês de uma data. Igual a **DAY**	DAYOFMONTH(*data*)	select dayofmonth('2023-01-15'); → retorna 15 select day(curdate));
DAYOFWEEK	Retorna o índice do dia da semana de 1 a 7. Ver **WEEKDAY**	DAYOFWEEK(*data*) 1=Sunday, 2=Monday, 3=Tuesday, 4=Wednesday, 5=Thursday, 6=Friday, 7=Saturday	select dayofweek('2023-01-15'); → retorna 5 select dayofweek(curdate());
DAYOFYEAR	Retorna o dia do ano de uma data	DAYOFYEAR(*data*)	select dayofyear('2023-06-15'); → retorna 166
EXTRACT	Extrai uma parte específica de uma data	EXTRACT(*unidade* FROM *data*) Unidades: *	select extract(month from '2023-01-15'); → retorna 1 select extract(year_month from '2017-06-15 09:34:21'); → retorna 201706
FROM_DAYS	Converte um número de dias em uma data. Oposto de **TO_DAYS**	FROM_DAYS(*número*)	select from_days(685467); → retorna 1876-09-29

HOUR	Retorna a hora de uma data/hora	HOUR(*datetime*) Máx 838h	select hour('10:45:00'); → retorna 10
LAST_DAY	Retorna o último dia do mês de uma data	LAST_DAY(*data*)	select last_day('2023-01-15'); → retorna 2023-01-31
LOCALTIME	Retorna data e hora locais atuais. Igual a **NOW** e **LOCAITIMESTAMP**	LOCALTIME() Retorna 'AAAA-MM-DD HH-MM-SS'(string) ou AAAAMMDD HHMMSS.uuuuuu (numérico)	select localtime();
LOCALTIMESTAMP	Retorna data e hora locais atuais. Igual a **NOW** e **LOCAITIME**	LOCALTIMESTAMP() Retorna 'AAAA-MM-DD HH-MM-SS'(string) ou AAAAMMDD HHMMSS.uuuuuu (numérico)	select localtimestamp();
MAKEDATE	Cria uma data a partir de ano e número de dias	MAKEDATE(*ano*, *dias*)	select makedate(2023, 32); → retorna 2023-02-01
MAKETIME	Cria um tempo a partir de hora, minuto e segundo	MAKETIME(*hora*, *minuto*, *segundo*) Máx 999999	select maketime(10, 30, 45); → retorna 10:30:45
MICROSECOND	Retorna os microssegundos de uma data/hora	MICROSECOND(*datetime*)	select microsecond('10:30:45.123456'); → retorna 123456
MINUTE	Retorna os minutos de uma data/hora	MINUTE(*datetime*)	select minute('10:45:00'); → retorna 45
MONTH	Retorna o mês de uma data	MONTH(*data*)	select month('2023-01-15'); → retorna 1
MONTHNAME	Retorna o nome do mês de uma data	MONTHNAME(data)	select monthname('2023-01-15'); → retorna January
NOW	Retorna data e hora atuais. Igual a **LOCAITIME** e **LOCAITIMESTAMP**	NOW() Retorna 'AAAA-MM-DD HH-MM-SS'(string) ou AAAAMMDD HHMMSS.uuuuuu (numérico)	select now();

PERIOD_ADD	Adiciona meses a um período (formato AAAAMM)	PERIOD_ADD(*período*, *meses*)	select period_add(202301, 2); → retorna 202303 select period_add(202301, -2); → retorna 202211
PERIOD_DIFF	Diferença entre dois períodos (formato AAAAMM)	PERIOD_DIFF(*período1*, *período2*) *Período*: AAMM ou AAAAMM	select period_diff(202303, 202301); → retorna 2 select period_diff(1703, 1612); → retorna 3
QUARTER	Retorna o trimestre de uma data	QUARTER(*data*) Janeiro-Março: retorna 1 Abril-Junho: retorna 2 Julho-Setembro: retorna 3 Outibro-Dezembro: retona 4	select quarter('2023-05-15'); → retorna 2
SECOND	Retorna os segundos de uma data/hora	SECOND(*datetime*)	select second('10:45:30'); → retorna 30
SEC_TO_TIME	Converte segundos em tempo	SEC_TO_TIME(*segundos*)	select sec_to_time(3661); → retorna 01:01:01 select sec_to_time(-6897); → retorna -01:54:57
STR_TO_DATE	Converte uma string em data, com formato especificado	STR_TO_DATE(*string*, *formato*) Formatos: **	select str_to_date('15/01/2023', '%d/%m/%Y'); → retorna 2023-01-15 select str_to_date("August 10 2017", "%M %d %Y"); → retorna 2017-08-10

SUBDATE	Subtrai um intervalo de tempo/data de uma data. Igual a **DATE_SUB**. Oposto de **DATE_SUB** e **DATESUB** e **ADDDATE**	SUBDATE(*data*, INTERVAL *valor unidade*)
		ou
		SUBDATE(*data*, *dias*)
		
		Valor: tanto positivo como negativo (negativo acrescenta, positivo retira)
		Unidades: *
			select subdate('2023-01-15', interval 5 day);
			→ retorna 2023-01-10
			
			select subdate('2023-01-15', interval -5 day);
			→ retorna 2023-01-20

SUBTIME	Subtrai um intervalo de tempo de uma data/hora	SUBTIME(*datetime*, *tempo*) *Valor*: tanto positivo como negativo (negativo acrescenta, positivo retira)	select subtime('10:30:00', '01:15:00'); → retorna 09:15:00 select subtime'"2017-06-15 10:24:21.000004', '5.000001'); → retorna 2017-06-15 10:24:16.000003 select subtime('10:24:21', '5'); → retorna 10:24:16 select subtime('10:24:21', '-3:2:5'); → retorna 13:26:26
SYSDATE	Retorna data e hora atuais (no momento da execução)	SYSDATE() Retorna 'AAAA-MM-DD HH-MM-SS'(string) ou AAAAMMDD HHMMSS.uuuuuu (numérico)	select sysdate();
TIME	Extrai a parte do tempo de uma data/hora	TIME(*datetime*)	select time('2023-01-15 10:30:00'); → retorna 10:30:00
TIME_FORMAT	Formata um tempo conforme especificado	TIME_FORMAT(*tempo*, *formato*) Formatos: **	select time_format('10:30:00', '%h:%i'); → retorna 10:30 select time_format('19:30:10', '%H %i %s'); → retorna 19 30 10

TIME_TO_SEC	Converte tempo em segundos	TIME_TO_SEC(*tempo*)	select time_to_sec('19:30:10'); → retorna 70210
TIMEDIFF	Diferença entre duas datas/horas	TIMEDIFF(*datetime1*, *datetime2*)	select timediff('10:30:00', '09:00:00'); → retorna 01:30:00
TIMESTAMP	Cria um datetime a partir de data ou datetime	TIMESTAMP(*data*, *hora*) *Hora*: opcional	select timestamp('2023-01-15', '10:30:00'); → retorna 2023-01-15 10:30:00 select timestamp('2023-01-15'); → retorna 2023-01-15 00:00:00
TO_DAYS	Retorna o número de dias desde "0000-00-00" até a data especificada. Oposto de **FROM_DAYS**	TO_DAYS(*data*)	select to_days('2023-01-15'); → retorna 738900
WEEK	Retorna o número da semana de uma data. Ver **WEEKOFYEAR**	WEEK(*data*, *1ºdiaSemana*) *1ºdiaSemana:* opcional 0 - 1º dia da semana é Domingo 1 - 1º dia da semana é Segunda e a 1ª semana do ano tem mais que 3 dias 2 - 1º dia da semana é Domingo 3 - 1º dia da semana é Segunda e a 1ª semana do ano tem mais que 3 dias 4 - 1º dia da semana é Domingo e a 1ª semana do ano tem mais que 3 dias 5 - 1º dia da semana é Segunda 6 - 1º dia da semana é Domingo e a 1ª semana do ano tem mais que 3 dias 7 - 1º dia da semana é Segunda	select week('2023-01-15'); → retorna 3
WEEKDAY	Retorna o número do dia da semana (0 = segunda, 6 = domingo). Ver **DAYOFWEEK**	WEEKDAY(*data*) 0=Monday, 1=Tuesday, 2=Wednesday, 3=Thursday, 4=Friday, 5=Saturday, 6=Sunday	select weekday('2025-08-17'); → retorna 6

WEEKOFYEAR	Retorna o número da semana do ano (1 a 53). Ver **WEEK**	WEEKOFYEAR(*data*)	select weekofyear('2025-08-17'); → retorna 33
YEAR	Retorna o ano de uma data	YEAR(*data*)	select year('2025-08-17'); → retorna 2025
YEARWEEK	Retorna o ano e o número da semana (formato: AAAASSS)	YEARWEEK(*data*, *1ºdiaSemana*) *1ºdiaSemana:* opcional 0 - 1º dia da semana é Domingo 1 - 1º dia da semana é Segunda e a 1ª semana do ano tem mais que 3 dias 2 - 1º dia da semana é Domingo 3 - 1º dia da semana é Segunda e a 1ª semana do ano tem mais que 3 dias 4 - 1º dia da semana é Domingo e a 1ª semana do ano tem mais que 3 dias 5 - 1º dia da semana é Segunda 6 - 1º dia da semana é Domingo e a 1ª semana do ano tem mais que 3 dias *7 - 1º dia da semana é Segunda*	select yearweek('2025-08-17'); → 202533

* Unidades:

microsecond
second
minute
hour
day
week
month
quarter
year
second_microsecond

minute_microsecond
minute_second
hour_microsecond
hour_second
hour_minute
day_microsecond
day_second
day_minute
day_hour
year_month

** Formatos:

Código	Descrição em Português
%a	Nome abreviado do dia da semana (Sun a Sat)
%b	Nome abreviado do mês (Jan a Dec)
%c	Mês como valor numérico (0 a 12)
%D	Dia do mês com sufixo ordinal (1st, 2nd, 3rd, ...)
%d	Dia do mês como valor numérico com dois dígitos (01 a 31)
%e	Dia do mês como valor numérico (0 a 31)
%f	Microssegundos (000000 a 999999)
%H	Hora no formato 24h com dois dígitos (00 a 23)
%h	Hora no formato 12h com dois dígitos (00 a 12)
%I	Hora no formato 12h com dois dígitos (00 a 12)
%i	Minutos (00 a 59)
%j	Dia do ano (001 a 366)
%k	Hora no formato 24h (0 a 23)
%l	Hora no formato 12h (1 a 12)
%M	Nome completo do mês (January a December)
%m	Mês como valor numérico com dois dígitos (00 a 12)
%p	AM ou PM
%r	Hora no formato 12h com AM/PM (hh:mm:ss AM/PM)
%S	Segundos (00 a 59)
%s	Segundos (00 a 59)
%T	Hora no formato 24h (hh:mm:ss)
%U	Semana do ano onde domingo é o primeiro dia (00 a 53)
%u	Semana do ano onde segunda-feira é o primeiro dia (00 a 53)
%V	Semana do ano (domingo como primeiro dia), usada com %X (01 a 53)
%v	Semana do ano (segunda-feira como primeiro dia), usada com %x (01 a 53)
%W	Nome completo do dia da semana (Sunday a Saturday)
%w	Dia da semana como número (domingo = 0, sábado = 6)
%X	Ano da semana (domingo como primeiro dia), usado com %V
%x	Ano da semana (segunda-feira como primeiro dia), usado com %v
%Y	Ano com quatro dígitos
%y	Ano com dois dígitos

▷ Funções avançadas

Função	Descrição	Sintaxe	Exemplo
BIN	Retorna a representação binária de um número. Ver CONV	BIN(*número*)	select bin(5); → retorna '101'
BINARY	Converte um valor para string binária (sensível a maiúsculas/minúsculas)	BINARY *valor* (importante para comparações case-sensitive	select binary 'abc'; → retorna 'abc' select * from utilizadores where binary nome='Ana': → retorna registos 'Ana' mas não 'ana' ou 'ANA'
CASE	Avalia condições e retorna o valor da primeira verdadeira	CASE WHEN *condição* THEN *resultado* ELSE *outroResultado* END	select case when 1=1 then 'sim' else 'não' end;
CAST	Converte um valor para um tipo de dado especificado. Ver CONVERT	CAST(*valor* AS *tipo*) *Tipo*: *	select cast('123' as unsigned); → retorna 123
COALESCE	Retorna o primeiro valor não nulo de uma lista	COALESCE(*valor1*, *valor2*, ...)	select coalesce(null, null, 'sandra'); → retorna 'sandra'
CONNECTION_ID	Retorna o ID único da conexão atual com o servidor	CONNECTION_ID()	select connection_id();
CONV	Converte um número entre sistemas de base numérica. Ver BIN	CONV(*número*, *base_origem*, *base_destino*)	select conv(10, 10, 2); → retorna '1010' select conv(1010, 2, 10); → retorna '10'

CONVERT	Converte um valor para tipo de dado ou conjunto de caracteres. Ver **CAST**	CONVERT(*valor*, *tipo*) ou CONVERRT(*valor* USING *charset*) *Tipo*:* *charset*: utf8 → suporta caracteres internacionais (acentos, emojis, etc.) latin1 → usado em idiomas europeus ocidentais ascii → apenas caracteres básicos (sem acentos)	select convert('123', unsigned); → retorna 123
CURRENT_USER	Retorna o nome e host do utilizador autenticado. Ver **USER**	CURRENT_USER()	select current_user();
DATABASE	Retorna o nome da base de dados atual	DATABASE()	select database();
IF	Retorna um valor se a condição for verdadeira, outro se for falsa	IF(*condição*, *valorSeVerdadeiro*, *valorSeFalso*)	select if(1=1, 'sim', 'não'); → retorna 'sim' select pedido_id, quantidade, if(quantidade>10, 'muitos', 'poucos') from pedidos;
IFNULL	Retorna um valor se a expressão for NULL, senão retorna a própria expressão	IFNULL(*expressão*, *valorSeNull*)	select ifnull(null, 'vazio'); → retorna 'vazio' select ifnull(nome_cliente, 'visitante') from clientes; → retorna 'visitante' se o nome for null
ISNULL	Retorna 1 se a expressão for NULL, senão 0	ISNULL(*expressão*) Null retorna 1 senão retorna 0	select isnull(null); → retorna 1
LAST_INSERT_ID	Retorna o ID gerado automaticamente (auto_increment) pela última inserção	LAST_INSERT_ID()	select last_insert_id();

NULLIF	Compara duas expressões e retorna NULL se forem iguais, senão retorna a primeira	NULLIF(*expr1*, *expr2*)	select nullif(5, 5); → retorna null select nullif(6, 5); → retorna 6
SESSION_USER	Retorna o nome e host do utilizador da sessão atual. Igual a **SYSTEM_USER** e **USER**. Ver **CURRENT_USER**	SESSION_USER()	select session_user();→ retorna 'root@localhost'
SYSTEM_USER	Retorna o nome e host do utilizador da sessão atual. Igual a **SESSION_USER** e **USER**. Ver **CURRENT_USER**	SYSTEM_USER()	select system_user(); → retorna 'root@localhost'
USER	Retorna o nome e host do utilizador atual. Igual a **SYSTEM_USER** e **SESSION_USER**. Ver **CURRENT_USER**. Ver **CURRENT_USER**	USER()	select user(); → retorna 'root@localhost'
VERSION	Retorna a versão atual do MySQL	VERSION()	select version();

* Tipo:

Valor	Descrição
DATE	Converte o valor para **DATA**. Formato: "AAAA-MM-DD"
DATETIME	Converte o valor para **DATA e HORA**. Formato: "AAAA-MM-DD HH:MM:SS"
DECIMAL	Converte o valor para **DECIMAL**. Pode usar os parâmetros opcionais **M** (máximo de dígitos) e **D** (dígitos após a vírgula).
TIME	Converte o valor para **HORA**. Formato: "HH:MM:SS"
CHAR	Converte o valor para **CHAR** (cadeia de caracteres de comprimento fixo)
NCHAR	Converte o valor para **NCHAR** (semelhante ao CHAR, mas usa o conjunto de caracteres nacional)
SIGNED	Converte o valor para **SIGNED** (inteiro de 64 bits com sinal)
UNSIGNED	Converte o valor para **UNSIGNED** (inteiro de 64 bits sem sinal)
BINARY	Converte o valor para **BINARY** (cadeia binária; útil para comparações sensíveis a maiúsculas/minúsculas e tratamento binário de dados)

▶ Funções MS Access

▷ Funções de string

Função	Descrição
Asc	Retorna o valor ASCII de um caractere específico
Chr	Retorna o caractere correspondente ao código ASCII especificado
Concat com &	Adiciona duas ou mais strings
CurDir	Retorna o caminho completo de uma unidade especificada
Format	Formata um valor com o formato especificado
InStr	Obtém a posição da primeira ocorrência de uma string dentro de outra
InstrRev	Obtém a posição da primeira ocorrência de uma string dentro de outra, a partir do final
LCase	Converte uma string para letras minúsculas
Left	Extrai um número de caracteres de uma string (a partir da esquerda)
Len	Retorna o comprimento de uma string
LTrim	Remove os espaços à esquerda de uma string
Mid	Extrai caracteres de uma string (a partir de qualquer posição)
Replace	Substitui uma substring dentro de uma string por outra, um número especificado de vezes
Right	Extrai um número de caracteres de uma string (a partir da direita)
RTrim	Remove os espaços à direita de uma string
Space	Retorna uma string com o número especificado de espaços
Split	Divide uma string em um array de substrings
Str	Retorna um número como string
StrComp	Compara duas strings
StrConv	Retorna uma string convertida
StrReverse	Inverte uma string e retorna o resultado
Trim	Remove os espaços à esquerda e à direita de uma string
UCase	Converte uma string para letras maiúsculas

▷ Funções numéricas

Função	Descrição
Abs	Retorna o valor absoluto de um número
Atn	Retorna o arco tangente de um número
Avg	Retorna o valor médio de uma expressão
Cos	Retorna o cosseno de um ângulo
Count	Retorna o número de registros retornados por uma consulta SELECT
Exp	Retorna *e* elevado à potência de um número especificado
Fix	Retorna a parte inteira de um número
Format	Formata um valor numérico com o formato especificado
Int	Retorna a parte inteira de um número
Max	Retorna o maior valor em um conjunto de valores
Min	Retorna o menor valor em um conjunto de valores
Randomize	Inicializa o gerador de números aleatórios (usado por Rnd()) com uma semente
Rnd	Retorna um número aleatório
Round	Arredonda um número para um número especificado de casas decimais

Sgn	Retorna o sinal de um número (positivo, negativo ou zero)
Sqr	Retorna a raiz quadrada de um número
Sum	Calcula a soma de um conjunto de valores
Val	Lê uma string e retorna os números encontrados na string

▷ Funções de data

Função	Descrição
Date	Retorna a data atual do sistema
DateAdd	Adiciona um intervalo de tempo/data a uma data e retorna o novo valor
DateDiff	Retorna a diferença entre duas datas
DatePart	Retorna uma parte específica de uma data (como um número inteiro)
DateSerial	Retorna uma data a partir dos valores especificados de ano, mês e dia
DateValue	Retorna uma data com base em uma string
Day	Retorna o dia do mês de uma determinada data
Format	Formata um valor de data com o formato especificado
Hour	Retorna a parte da hora de um valor de tempo/data
Minute	Retorna os minutos de um valor de tempo/data
Month	Retorna o mês de uma determinada data
MonthName	Retorna o nome do mês com base em um número
Now	Retorna a data e hora atuais com base no sistema do computador
Second	Retorna os segundos de um valor de tempo/data
Time	Retorna a hora atual do sistema
TimeSerial	Retorna um valor de hora a partir dos valores de hora, minuto e segundo
TimeValue	Retorna um valor de hora com base em uma string
Weekday	Retorna o número do dia da semana de uma determinada data
WeekdayName	Retorna o nome do dia da semana com base em um número
Year	Retorna o ano de uma determinada data

▷ Outras funções

Função	Descrição
CurrentUser	Retorna o nome do usuário atual da base de dados
Environ	Retorna uma string com o valor de uma variável de ambiente do sistema operacional
IsDate	Verifica se uma expressão pode ser convertida em uma data
IsNull	Verifica se uma expressão contém *Null* (sem dados)
IsNumeric	Verifica se uma expressão é um número válido

▶ Funções SQL Server

▷ Funções de string

Função	Descrição
ASCII	Retorna o valor ASCII de um caractere específico
CHAR	Retorna o caractere com base no código ASCII
CHARINDEX	Retorna a posição de uma substring dentro de uma string
CONCAT	Adiciona duas ou mais strings
Concat com +	Adiciona duas ou mais strings usando o operador +
CONCAT_WS	Adiciona duas ou mais strings com um separador
DATALENGTH	Retorna o número de bytes usados para representar uma expressão
DIFFERENCE	Compara dois valores SOUNDEX e retorna um valor inteiro
FORMAT	Formata um valor com o formato especificado
LEFT	Extrai um número de caracteres de uma string (a partir da esquerda)
LEN	Retorna o comprimento de uma string
LOWER	Converte uma string para letras minúsculas
LTRIM	Remove os espaços à esquerda de uma string
NCHAR	Retorna o caractere Unicode com base no código numérico
PATINDEX	Retorna a posição de um padrão dentro de uma string
QUOTENAME	Retorna uma string Unicode com delimitadores para torná-la um identificador válido no SQL Server
REPLACE	Substitui todas as ocorrências de uma substring por outra
REPLICATE	Repete uma string um número especificado de vezes
REVERSE	Inverte uma string e retorna o resultado
RIGHT	Extrai um número de caracteres de uma string (a partir da direita)
RTRIM	Remove os espaços à direita de uma string
SOUNDEX	Retorna um código de quatro caracteres para avaliar a similaridade entre duas strings
SPACE	Retorna uma string com o número especificado de espaços
STR	Retorna um número como string
STUFF	Exclui parte de uma string e insere outra parte, começando em uma posição específica
SUBSTRING	Extrai caracteres de uma string
TRANSLATE	Traduz os caracteres de uma string com base em dois conjuntos de substituição
TRIM	Remove espaços (ou outros caracteres especificados) do início e fim de uma string
UNICODE	Retorna o valor Unicode do primeiro caractere de uma expressão
UPPER	Converte uma string para letras maiúsculas

▷ Funções numéricas

Função	Descrição
ABS	Retorna o valor absoluto de um número
ACOS	Retorna o arco cosseno de um número
ASIN	Retorna o arco seno de um número
ATAN	Retorna o arco tangente de um número
ATN2	Retorna o arco tangente de dois números

AVG	Retorna o valor médio de uma expressão
CEILING	Retorna o menor número inteiro maior ou igual ao número especificado
COUNT	Retorna o número de registros retornados por uma consulta SELECT
COS	Retorna o cosseno de um número
COT	Retorna a cotangente de um número
DEGREES	Converte um valor em radianos para graus
EXP	Retorna *e* elevado à potência de um número especificado
FLOOR	Retorna o maior número inteiro menor ou igual ao número especificado
LOG	Retorna o logaritmo natural ou o logaritmo de um número em uma base específica
LOG10	Retorna o logaritmo de base 10 de um número
MAX	Retorna o maior valor em um conjunto de valores
MIN	Retorna o menor valor em um conjunto de valores
PI	Retorna o valor de π (pi)
POWER	Retorna o valor de um número elevado à potência de outro número
RADIANS	Converte um valor em graus para radianos
RAND	Retorna um número aleatório
ROUND	Arredonda um número para o número especificado de casas decimais
SIGN	Retorna o sinal de um número (positivo, negativo ou zero)
SIN	Retorna o seno de um número
SQRT	Retorna a raiz quadrada de um número
SQUARE	Retorna o quadrado de um número
SUM	Calcula a soma de um conjunto de valores
TAN	Retorna a tangente de um número

▷ Funções de data

Função	Descrição
CURRENT_TIMESTAMP	Retorna a data e hora atual
DATEADD	Adiciona um intervalo de tempo/data a uma data e retorna o novo valor
DATEDIFF	Retorna a diferença entre duas datas
DATEFROMPARTS	Retorna uma data a partir dos valores especificados de ano, mês e dia
DATENAME	Retorna uma parte específica da data (como texto)
DATEPART	Retorna uma parte específica da data (como número inteiro)
DAY	Retorna o dia do mês de uma data especificada
GETDATE	Retorna a data e hora atual do sistema do banco de dados
GETUTCDATE	Retorna a data e hora atual em formato UTC do sistema do banco de dados
ISDATE	Verifica se uma expressão é uma data válida (retorna 1 se for, senão 0)
MONTH	Retorna o mês de uma data especificada (número de 1 a 12)
SYSDATETIME	Retorna a data e hora atual do SQL Server com maior precisão
YEAR	Retorna o ano de uma data especificada

▷ Funções avançadas

Função	Descrição
CAST	Converte um valor (de qualquer tipo) para um tipo de dado especificado
COALESCE	Retorna o primeiro valor não nulo em uma lista

CONVERT	Converte um valor (de qualquer tipo) para um tipo de dado especificado
CURRENT_USER	Retorna o nome do usuário atual no banco de dados do SQL Server
IIF	Retorna um valor se a condição for VERDADEIRA, ou outro valor se for FALSA
ISNULL	Retorna um valor especificado se a expressão for NULL, senão retorna a expressão
ISNUMERIC	Verifica se uma expressão é numérica
NULLIF	Retorna NULL se duas expressões forem iguais
SESSION_USER	Retorna o nome do usuário atual na sessão do SQL Server
SESSIONPROPERTY	Retorna as configurações da sessão para uma opção especificada
SYSTEM_USER	Retorna o nome de login do usuário atual
USER_NAME	Retorna o nome do usuário do banco de dados com base no ID especificado